File I/O in C++

The C-Style Approach: Mastering fprintf & fscanf

Why Use File I/O?

Persistence: Standard input/output (console) is volatile. Files allow data to exist beyond the program's execution.

Volume: Essential for processing large datasets (10,000+ lines) common in Data Structures & Algorithms.

Automation: Allows your code to read test cases automatically without manual typing.

The C++ Setup

1. The Header

In C++, we use the header to access C-style input/output functions. This is the C++ equivalent of C's .

```
#include
```

The C++ Setup

2. The Handle

We manipulate files using a **File Pointer**. It acts as a handle to the file stream.

```
FILE *fp;
```

Note: fp is just a variable name, but the type must be FILE *.

Opening a File

Syntax: fopen

Use the fopen function to initialize the file pointer.

```
fp = fopen("data.txt", "r");
```

Common Modes:

- "r" : **Read** (File must exist)
- "w" : **Write** (Creates new file or overwrites)
- "a" : **Append** (Adds to the end of file)

Writing to Files

Function: fprintf

Think of it as "File Print Formatted". It works exactly like printf,

but takes the file pointer as the first argument.

It sends formatted text directly into the file stream you opened

with "w" or "a" mode.

fprintf in Action

Code Example

```
int score = 95;
char name[] = "Alice";

// Write to file
fprintf(fp, "Student: %s, Score: %d\n" ,
name, score);
```

fprintf in Action

Placeholders

The format specifiers are identical to standard C:

- %d : Integer
- %f : Float / Double
- %s : String (char array)
- %c : Single Character

Reading From Files

Function: fscanf

Think of it as "File Scan Formatted". It reads data from the file stream into your variables.

Crucial: Like scanf, you must provide the *address* of the variable using & (except for arrays/strings).

fscanf in Action

Reading One Item

```
int id;
// Read an integer from file
fscanf(fp, "%d", &id);
```

Don't forget the **&** !

fscanf in Action

Reading Until End

```c
int num;
// Loop until End Of File (EOF)
while (fscanf(fp, "%d", &num) ≠ EOF) {
  printf("Read: %d\n", num);
}
```

Safety First: Checking for Errors

The Risk

If a file doesn't exist (in 'r' mode) or permission is denied, fopen will fail.

The Signal

On failure, fopen returns NULL. You must check for this before using the pointer.

Safety First: Checking for Errors

The Fix

```c
if (fp == NULL) {
  printf("Error opening file!\n");
  return 1;
}
```

Closing the File

Syntax: fclose

Always close your file when you are done.

```
fclose (fp);
```

Why?

- **Flushes Data:** Ensures all data is physically written to the disk.
- **Frees Resources:** Releases the memory used by the file pointer.
- **Prevents Corruption:** Safely disconnects the program from the file.

The Complete Workflow

1. **Include**
2. **Open file with fopen**
3. **Check for NULL**
4. **Read/Write with fscanf/fprintf**
5. **Close with fclose**

Questions?